

MME8215: Operations Research

Mixed Integer Programming

Contents

1 Combinatorial Optimization	1
1.1 Introduction	1
1.2 Types of techniques	2
1.3 Efficiency of Techniques	3
1.3.1 Classification of problems in complexity theory	4
2 Mixed integer Programming	6
2.1 Introduction	6
2.2 Set up	6
2.3 Examples	7
2.3.1 Knapsack Problem	7
2.3.2 Assignment problems	8
2.3.3 Sequencing Problems	8

Next we cover basic theoretical and practical aspects of Mixed integer Programming (MIP) both theoretical and practical topics related to Continuous Linear Programming (CLP) and Network Theory. Like, Continuous Linear Programming, MIP uses linear equations to model optimization problems. However, some of the decision variables can only take Integer values. Feasible solutions are no longer found at vertices of the feasibility region, and this fundamental fact drastically increases the complexity of MIP problems. Network Theory provides a framework rather useful to solve MIP problems, and therefore it is also introduced in this course. CLP allows us to model, solve and analyse the solution of optimization problems using linear equations. “The world is not linear” but still, CLP is a really powerful tool to solve a plethora of real world optimisation problems.

1 Combinatorial Optimization

1.1 Introduction

Combinatorial optimization gathers different types of optimization problems where the decision maker needs to select an alternative from a finite set of possible alternatives with different outcomes, given a set of requirements (i.e. constraints). Some examples of problems in combinatorial optimization are:

- **Grouping:** Find a subset of members within a set that optimizes an objective function (e.g. minimises costs or maximises profits) given a set of constraints. For instance, select the best starting 11 in a football team given their performance statistics.
- **Ordering:** Find the optimal ordering of the members of a set that maximise or minimise an objective function with some constraints. For instance the optimal sequence of production orders that minimises production time.
- **Assignment:** Assign members of a set to members of another set to optimise an objective function given a set of constraints. For instance, assign tasks to employees at a minimum cost

Combinatorial optimization is strongly connected with set theory and the concept of search, or (iteratively) explore the set to find the optimal solution. Heuristics will explore the set of alternatives looking for the optimal solution.

The following types of problems are outstanding instances of combinatorial optimization problems:

- **Integer Programming:** Any Linear Programming Problems with Integer or binary variables. When this happens, the feasibility region and objective function are no longer convex, and we cannot apply the same principles and methods as with CLP. When we combine Integer, Binary and Continuous variables, we are dealing with Mixed Integer Programming (MIP). MIP covers a wide range of OR problems, which we will be covering extensively in this unit.
- **Travelling salesman problems:** In the classic formulation of the Travelling Salesman Problem (TSP), a salesman needs to visit a set of cities and wants to find the optimal route that would take him through all cities and back to his/her original departure city. Thus, given a list of cities and the distances between each pair of cities, the TSP aims to determine what is the shortest possible route that visits each city and returns to the origin city. The TSP is thus an ordering problem. There are many OR problems, specially in logistics and manufacturing that can be modeled as TSP.
- **Knapsack problems:** In knapsack problems, given a set of items, each with a weight and a value, we need to determine the number of each item to include in a collection so that the total value is maximised while the total weight is less than or equal than the maximum weight the knapsack can hold.
- **Minimum Spanning tree:** Minimum Spanning Trees (MST) problems have a wide number of applications in fields like logistics or telecommunications when we deal with networks that interconnect different objects or edges through vertices. In this context, MST are grouping problems where we want to find a subset of edges in a graph that interconnects certain vertices at a minimum cost. We will address MST problems, among others, in Network Theory.

As you can already imagine by now, there is a really wide range of applications for combinatorial optimization. Some examples are Planning and Scheduling of activities, product and process design, portfolio selection, networking and routing (Internet traffic, airlines routes, logistics).

1.2 Types of techniques

Given the importance of the field, scientist have developed many techniques to solve combinatorial optimization problems over the years. They can be classified into the following categories:

- **Numeric Techniques:** Numeric techniques gather algorithms that provide an optimal solution by exploring all possible options. The simplest example is brute force, although there are many other algorithms that fall into this category. Whenever you use numeric functions like Python's Numpy to compute a minimum, maximum, or ordering of a set, internally the library is applying a numeric technique to compute the result. As we will soon see, for problem instances where the set is large, it might not be computationally efficient to rely solely on numeric techniques, and that is why we have other methods. Numeric techniques are out of the scope of this unit.
- **Problem-solving heuristics:** These techniques find a (sub-)optimal solution in a given number of steps. We will focus on some of these techniques in this unit. Some examples are:
 - **State space search:** These algorithms test successive combinations (or states) of the decision variables iteratively, in such a way that at every step the solution is closer to the optimal solution, until the optimum solution is found. One example of search heuristic is the Branch and Bound algorithm that will be covered in this unit.
 - **Greedy algorithms:** Greedy algorithms first sort the members of the set according to their impact in the objective function, and add them to the solution iteratively.
 - **Genetic algorithms:** Explore the possible solutions combining the features of an initial set of possible solutions, selecting the fittest candidates to combine their features in the next iteration.
 - **Graph Theory:** Graph theory is not really a type of technique, but rather, a separate field of mathematics dedicated to the study of relations between objects through mathematical structures known as graphs. Many combinatorial optimization problems can be easily modeled through these structures, and there are many efficient algorithms that effectively compute graph-like structures to find MSTs or shortest paths which provide the optimal solution to our problem.

1.3 Efficiency of Techniques

But why do we need this plethora of techniques in the first place? The main reason is that it is not feasible to use numeric techniques in relatively large instances of problems, because the number of possible combinations grows exponentially with the number of members in the sets. The image below, taken from Dynamic logic by Harel et al., 2000¹ highlights this fact:

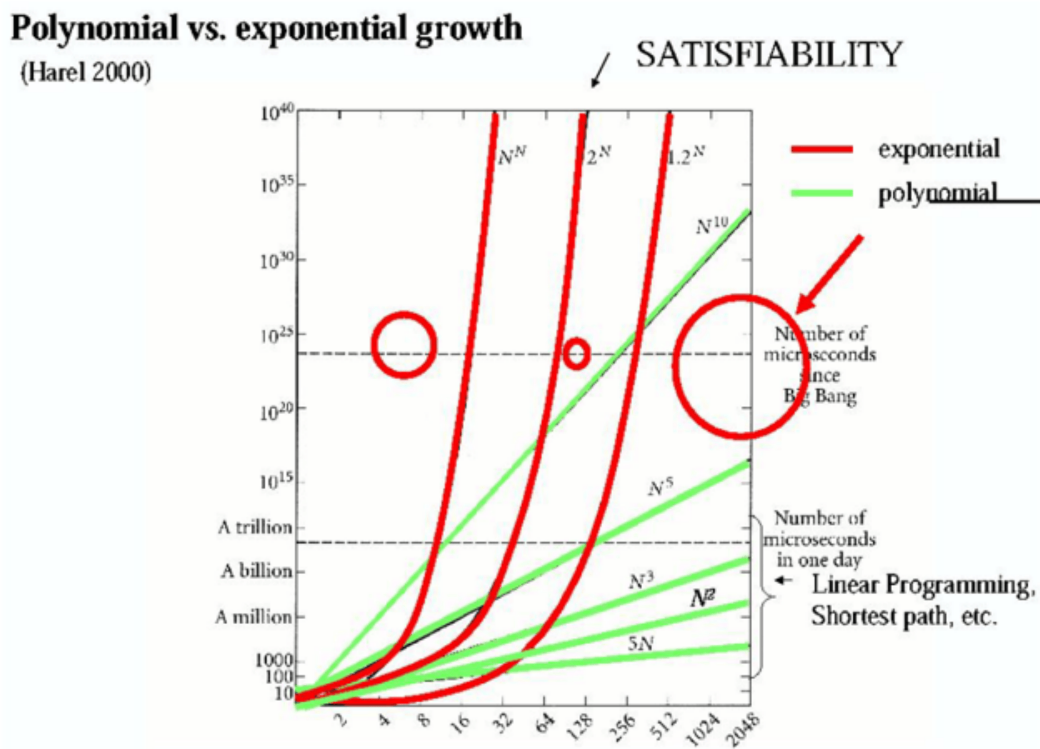
As shown in the graph, for relatively medium instances of combinatorial optimization problems like Integer Programming, the time required to explore all possible combinations might be larger than time itself! We need techniques that need fewer operations to find an optimal solution, or that at least can provide a suboptimal solution in a given interval of time.

Not all techniques provide results in the same number of operations, that is, not all techniques are equally efficient. An informal definition of an efficient algorithm is an algorithm in which the time required to solve a problem instance does not increase too fast as the problem size increases. Below, there is a formal definition of efficiency:

Definition 1 *An algorithm is efficient (good, polynomial) if the time necessary to solve any instance of input size L is bounded above by a polynomial function of L . (Thus it does not increase as fast as an exponential function such as s^L .)*

The Big $\mathcal{O}$ Notation is very handy to compare the efficiency of algorithms. For instance, an algorithm that solves a problem of size L in a time proportional to the square of L is a polynomial

¹David Harel, Dexter Kozen, Jerzy Tiuryn. 2000 Dynamic Logic. MIT Press.



algorithm in $\mathcal{O}(n^2)$, where as an algorithm that solves the same problem in an exponential time proportional to 2^L is noted as $\mathcal{O}(2^L)$. The latter is less efficient because the time required to solve the problem increases faster as L becomes larger. An important consideration that needs to be made at this point is that efficiency does not explicitly mention the number of operations, but rather the computing time. This has different implications. The first is that algorithms are formulated in mathematical language, and we need a computing model to translate operations into computational time. In complexity theory, which is the field of mathematics dedicated to the analysis of the complexity of problems, the Turing Machine, which is a mathematical model of a “perfect” machine is used to define time as a function of the number of operations in the definitions above of efficiency above.

Based on this definition of a computational machine model, problems can be classified according to their complexity, in a classification or taxonomy of problems.

1.3.1 Classification of problems in complexity theory

Based on the definitions above, a problem is as complex as the most efficient algorithm devised to solve it. An algorithm is considered efficient if it can solve an instance of a problem of input size L in a polynomial function of L . But what about other problems? Are there different categories of complex problems? Complexity theory is a field of mathematics that focuses precisely on the analysis of the complexity of problems, and its findings help scientist guide the search for new algorithms and mathematical techniques. The following sections describe the different problem classes, which are categories defined for problems according to their complexity.

Class Polynomial - Class P (Easy problems) A problem is in the class P (or polynomially solvable) if it admits a polynomial algorithm. This category gathers all problems for which we have found an algorithm which is able to find an efficient algorithm, able to solve it in a number of operations which is a polynomial function of the size of the problem.

Class Non-deterministic Polynomial time - Class NP (Possibly easy problems) A problem is in the class non-deterministic polynomial (NP) if the problem can be verified in polynomial time, meaning that we can find a bound for the optimal solution in polynomial time. Think for instance of a maximization problem, if we can find an upper bound for the objective function in polynomial time, we might not know which is the optimal solution, but we can already use this information in our problem, as we know that the optimal value will not exceed the upper bound.

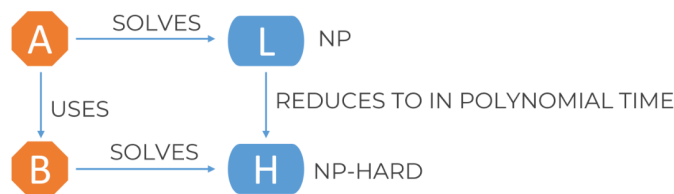
From this, we can define the associated decision problem as a problem that allows us to verify if a given number is a valid bound for our problem. If the result of the associated decision problem is True (meaning, yes, the provided number is a valid bound), then we can use this information to provide an approximation to the optimal solution.

Problems for which there is an algorithm that solves the associated decision problem in polynomial time are classified as NP problems.

Class Non-deterministic Polynomial time Hard - Class NP-Hard (Possibly Hard Problems) Reduction

To understand the NP-Hard category, we first need to introduce the concept of reduction. A reduction is basically an algorithm that allows us to convert an instance of a problem A into instances of another problem B. Formally, a problem H is NP-Hard if there is a Polynomial algorithm to reduce every problem L in class NP to it.

Let us use the following figure to illustrate this:



If a problem L can be reduced to H in polynomial time, and we can find a solution for H using an algorithm B in one unit of time, then we can solve L in polynomial time with an algorithm A which basically reduces L to H and then uses B to solve the problem.

The Traveling Salesman Problem (TSP) is an example of an NP-Hard problem.

Informal definition Based on the definition above, we can argue that NP-Hard problems are at least as hard as the hardest NP problem, since we can reduce every problem in NP to it in polynomial time, it follows that algorithms for NP-Hard problems are at least as efficient as algorithms for NP problems.

Class Non-Deterministic Polynomial time class Complete - NP-Complete (Hard Problems)

Formally, a problem is NP-Complete if:

- A given solution can be verified in polynomial time (that is, it is an NP Problem).
- It can be used to simulate any other problem for which we can verify that a solution in polynomial time (that is, it is an NP-Hard problem).

It follows that, if a problem is both NP and NP-Hard, it can be used to validate the solution of the complete NP class

Note however that although solutions for NP-Problems can be verified in polynomial time, this does not imply that an optimal solution can be found in polynomial time. That is, we can use heuristics to verify a bound and use this bound as an approximation of the optimal solution, and

we can do this efficiently in polynomial time, but we cannot find an optimal solution in polynomial time, which would in turn imply that the problem is in class P.

2 Mixed integer Programming

2.1 Introduction

Mixed Integer Programming (MIP) can be regarded as an extension of Continuous Linear Programming where any of or all the defined decision variables cannot take fractional values. MIP introduces two new types of variables:

- **Integer variables:** Integer variables can only take Integer values. Normally, Integer variables are introduced by logical restrictions that make rational valuables unfeasible. For instance, in some instances of production mix problems, it does not make sense to produce fractional parts of a product and treating them as real number is just an approximation.
- **Binary variables:** Binary variables or Boolean variables can only take values 1 or 0. Normally, binary variables are used to factor into our model binary decisions, e.g. to decide whether to use a machine configuration or another, or whether to select one element from a set or not.

In CLP, the continuity of the decision variables ensures that the feasibility region and the objective function intersect in a vertex of the feasible region at the optimal value. But this is no longer the case when some decision variables are Integer values. Intuitively, one can think that the optimal value may be found in one of the feasible Integer values around the theoretical intersection, as long as the solution is feasible. This makes the number of candidate solutions larger and consequently, MIP problems are more difficult to solve. From a mathematical point of view, when the decision variables are not continuous, the feasible region and the objective function are no longer convex. Convexity of feasibility region and objective function ensures that the optimal solution is at one vertex. We will explain this concept further in Non-Linear Programming. For now, it is sufficient to understand that in MIP, the feasibility region is no longer convex since not all values are possible and this makes it harder to find a solution.

This limitation makes convergence to an optimal solution more complex. MIP solution is several orders of magnitude more complex than linear programming. For instance, the assumptions made to design the Simplex algorithm do no longer hold and although adaptations may find an optimal solution, convergence is so slow that they cannot be applied to practical problems.

2.2 Set up

In general, the set-up of MIP problems is the same as the set-up of CLP problems, except that some decision variables can only take Integer values. For instance:

Decision variables Let us note our decision variable as $X = [x_1, x_2, \dots, x_n]$ where n is the number of decision variables. In continuous linear programming, the decision variables could take any real value, in mathematical notation this is expressed as: $x_j \in \mathbb{R}$.

Now, in *integer programming*, decision variables can only take integer numbers. Without loss of generalization, let us consider that our decision variables can only take positive integers, or natural numbers. We refer to this type of decision variables as *integer variables*, and are noted as: $x_j \in \mathbb{N}$.

A special type of integer variables are binary variables which are variables that can only take values 0, or 1. These variables are very convenient to model 'yes or no' decisions or 'true or false'

(boolean) logic decisions into our problem. We note can use this notation for binary variables: $x_j \in [0, 1]$.

In general, not all decision variables have to be necessarily of the same type. Integer Programming (IP) problems only have integer or binary decision variables, whereas MIP problems can have any combination of decision variables.

Objective function As with CLP, our objective function is a linear function of the decision variables, noted as z and which we would like to maximize or minimize. For instance, if our objective function is of type maximize, we can use:

$$\max z = \sum_{j=1}^n c_j \cdot x_j.$$

Constraints The objective function will be subject to some limitations or constraints:

subject to

$$\sum_{j=1}^n x_j \cdot a_{ij} \leq b_i, \forall i \in [1, m]$$

2.3 Examples

Let us now see some prominent examples of Integer Programming types. The next sections cover the formulation of some well known combinatorial optimization problems as IP problems.

2.3.1 Knapsack Problem

The Knapsack problem is an optimization problem that is often found in practical applications. The classical formulation reads

Definition 2 *Given a set of items, each with a weight and a value, we need to determine the number of each item to include in a collection so that the total value is maximised while the total weight is less than or equal than the maximum weight the knapsack can hold.*

Decision variables The decision variables x_j are binary decision variables that determine whether we include item j in the Knapsack: $x_j \in [0, 1]$, 0: item not included, 1: item included.

That is, the decision variable can only take values 0, or 1. The value 0 means that the object is not selected and therefore has no impact in the objective function. Likewise, the value 1 means that the object is selected.

Objective coefficient Each item j will have a different value v_j . We can express the objective function as the value of the items selected: $\max z = \sum_{j=1}^n v_j \cdot x_j$.

Constraints Now, our only constraint is related to the maximum weight W the knapsack can support. Each item will have a different weight w_j , and the sum of the weights w_j of the selected variables has to be less or equal than the maximum weight.

$$\sum_{j=1}^n w_j \cdot x_j \leq W$$

Note that, if a given x_j is zero, the product $w_j \cdot x_j$ will also be zero and will have no impact on the total value of the left hand side summation.

Now, it is important to note that in this formulation, weights and values as abstract concepts, and the same formulation can be applied to many practical problems.

2.3.2 Assignment problems

Assignment problems are a fundamental class of combinatorial optimization problems. In essence, assignment problems consists of assigning elements of a set of resources R (like workers, machines, etc.) to a set of tasks T to optimize a certain objective (like minimizing cost or maximizing efficiency), subject to various constraints.

Indices Now, since we have two different sets, we will use two different indices to uniquely identify the elements in each set:

- **i:** We will use index i to identify each element in set R . Let us note the size of set R as m , therefore, $i \in [1, 2, \dots, m]$
- **j:** We will use index j to identify each element in set T . The size of set T is noted as n , therefore, $j \in [1, 2, \dots, n]$

Decision Variables Let us define a binary variable x_{ij} for each resource i and task j . $x_{ij} = 1$ if resource i is assigned to task j , and 0 otherwise: $x_{ij} \in [0, 1]$ 0: resource i is not assigned to task j , 1: otherwise $\forall i, \forall j$.

Note that there are a total of $m * n$ decision variables!

Objective function The objective is to minimize costs or maximize benefits associated with assigning a particular resource to a particular task. As with Knapsack problems, we need to think of costs or benefits as abstract concepts, and the model can be applied to many practical applications.

For instance, let us note as c_{ij} the cost of assigning resource i to task j . The objective function then becomes:

$$\min z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} * x_{ij}.$$

Constraints Now, the constraints need to ensure that one element of R is assigned to one element T , that is, we need to ensure that there are not two or more resources assigned to a task:

$$\sum_{j=1}^n x_{ij} = 1, \forall i.$$

Note first that the for a given i , the left hand side $\sum_{j=1}^n x_{ij}$ is the sum of all the decision variables associated to a resource i . Since the different x_{ij} can only be 0 or 1, by establishing an upper bound limit of 1 to the sum of all the elements, we ensure that the resource can only be assigned at most to one task.

Similarly, we need to ensure that a task is only associated to one resource:

$$\sum_{i=1}^m x_{ij} = 1, \forall j.$$

Note that the expression is similar, but now the left hand side $\sum_{i=1}^m x_{ij}$ represents the sum of all the decision variables associated to task j .

2.3.3 Sequencing Problems

Traveling Salesman Problem Recall the classic formulation of the Travelling Salesman Problem (TSP):

Definition 3 *Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city.*

In practical applications, cities represent different steps in a process, and distances represent the cost of a sequence of steps in the process, from one step to another.

Thus, for the setup of the TSP as an IP, we have the following variables and coefficients:

- x_{ij} : Binary decision variables that represent a sequence from source i to destination j . 1 means that the step is selected in the final sequence, 0 otherwise.
- c_{ij} : Represents the cost the step in the sequence, in the classical formulation, the cost of travelling from city i to city j .
- u_i : Integer variable that represents the step of i in the sequence. If u_i is equal to y then city i is departed at step t .

The IP formulation is then as follows:

$$\min z = \sum_{i=1}^N \sum_{j=1}^N c_{ij} * x_{ij}.$$

Subject to: we can only depart from 1 city:

$$\sum_{j=1}^N x_{ij} = 1, \forall i.$$

We can only arrive to a city from a city:

$$\sum_{i=1}^N x_{ij} = 1, \forall j.$$

We cannot travel in loops in a city:

$$x_{ij} = 0, \forall i = j.$$

We travel from city i to city j exactly at step u_i :

$$u_i - u_j + n * x_{ij} = n - 1, \forall i \in 1 \leq i \neq j \leq n.$$

There are no loops, we do not allow u_i to be arbitrarily large:

$$0 \leq u_i \leq n, \forall i \in 2 \leq i \leq n.$$

MME8215: Operations Research

IX. Non-Linear Programming

Contents

1 Introduction	1
2 Unconstrained Non-Linear Programming	4
2.1 One dimensional problems set-up	4
2.2 Multidimensional problems set-up	4
3 Gradient Descent Methods	6

This course is an introduction to Non-Linear Programming (NLP), the branch of optimisation that deals with problem models where the functions that define the relationship between the unknowns (either objective function or constraints) are not linear. The fact that the functions are not linear makes NLP a lot harder to deal with from a mathematical point of view. We will focus on specific cases that have many applications in real life problems, particularly quadratic functions and their relationship to machine learning.

1 Introduction

A non-linear programming (NLP) problem is similar to a linear programming problem in the sense that it is also composed of an objective function and constraints, but a NLP includes at least a non-linear function. We will deal with continuous decision variables and problems where either the objective function, or any of the constraints are not linear. NLP problems are in many cases more difficult to solve than linear programming problems. The mathematical foundations of NLP optimisation date back to the work of Joseph Louis Lagrange. He developed a method to take into account non-linear constraints into optimisation problems, which is called Lagrange multipliers, and that will be used in constrained NLP.

Hard to distinguish local and global optimal values

In linear programming, methods like the simplex need only to know the value of the unknowns and the value of the objective function to determine if the solution is optimal, because the maximum is always at a vertex of the feasibility region. Non-linear functions have critical points, which can be either local or global maxima or minima:

- *Local maximum*: The value of the function at the critical point is higher than the values of the function around the critical point.
- *Global maximum*: The value of the function at the critical point is the highest value of the function.
- *Local minimum*: The value of the function at the critical point is lower than the values of the function around the critical point.
- *Global minimum*: The value of the function at the critical point is the lowest value of the function.

For instance, consider the function presented in figure 1. The function has many critical points. Any critical point can be a local maximum or a global maximum depending on the range of values that x can take. For instance, if x could only take values in the range $x \in [-3, 0]$, then the local maximum would become a global maximum. This makes it somewhat more difficult to find the optimal solution in an optimisation problem.

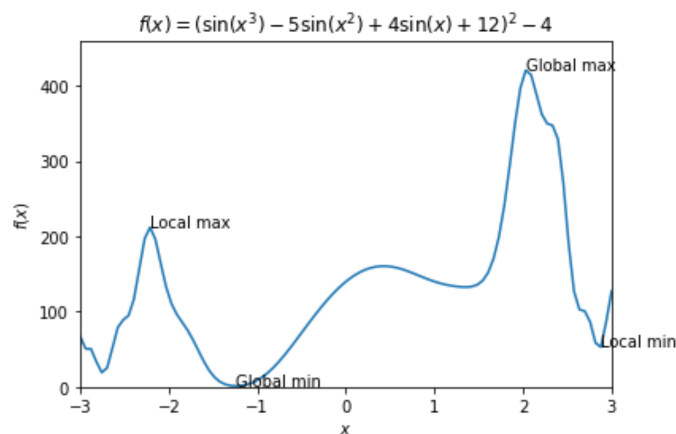


Figure 1: An example of a function with many critical points.

Optima not restricted to vertex

With non-linear functions, the optimal (maximum or minimum) value might no longer be at one of the vertices. For instance, let us take as an example the function presented in figure 2.

The example shows a non-linear function on two decision variables, x and y . The black lines represent some constraints limiting the feasibility region. The contour graph shows that the maximum value of the objective function is within the interior of the feasibility region and not in one of the vertices.

The following situations are examples where the optimal value might not be at one vertex of the feasibility region:

- *Multimodal function*: As in the example from figure 2, the function might be multimodal and depend on several decision variables, increasing its complexity.
- *Interior optima*: As in the above example, the optimal value might be found in an interior point of the feasibility region.
- *Discontinuous*: The objective function might be discontinuous and again, the optimal value might not be surely located in one vertex.

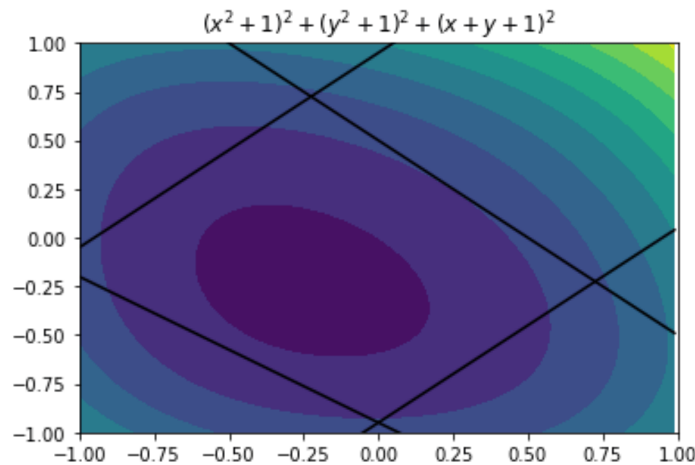


Figure 2: A non-linear function on two decision variables.

Given that the objective functions and the constraints can be of any family of functions, potentially, there is a huge body of mathematical theory that can be applied to NLP, since non-linear functions have a wide range of characteristics. The theoretical framework is more sophisticated, and modeling is also more difficult, since the researcher needs to define the types of function to use in the model.

Solver selection

Due to the complex mathematical theory behind, selecting the right solver to solve a specific problem can be complex. In fact, different algorithms and solvers can provide different results. Normally, algorithms deal with local optimisation, rather than global optimisation, and for that reason, the selection of the initial point used to start the algorithm might have an impact in the solution.

Hence, different algorithms might provide different solutions, but when a specific algorithm is selected, its configuration (particularly the initial point) might yield different results.

Types of problems

We can divide NLP into two different classes of problems with increasing complexity:

- *Unconstrained NLP Problems:* This class of NLP problems have no constraints, we look for a maximum or minimum of the objective function within some bounded decision variables, but not subject to any constraint.
- *Constrained NLP Problems:* This class of problems have constraints, i.e. there are restrictions to the objective function solution determined by a set of functions of the decision variables.

Additionally, to better present the mathematical framework behind NLP, we will consider One-Dimensional unconstrained problems, where there is only one decision variable, and Multi - Dimensional or multi-modal, where there are several decision variables

2 Unconstrained Non-Linear Programming

2.1 One dimensional problems set-up

Let us first consider just one decision variable. The objective is to search a maximum or minimum of a non-linear function. Without loss of generalisation, let us consider that the objective is to minimize the objective function:

$$\min f(x), x \in \mathbb{R}.$$

Any (local or global) minimum of $f(x)$, denoted as x^* , will have the following properties:

1. **First derivative test:** The first derivative of a function at a point x^* represents the slope of a line tangent to $f(x)$ at x^* . In a minimum, the first derivative is equal to zero, $\frac{df(x)}{dx} = 0$, meaning that the tangent is horizontal.
2. **Second derivative test:** The second derivative is positive $\frac{d^2f(x)}{dx^2}$, meaning that the slope of the tangent is increasing and therefore, it must be less than zero for $x < x^*$, meaning that the function decreases before x^* , and greater than zero for $x > x^*$, meaning that the function increases after x^* . Therefore $f(x)$ takes its minimum value at x^* , at least in its proximity.

By definition, x^* is a critical point. For a maximum, the second derivative is negative and using the same logic, $f(x)$ decreases in the proximity of x^* .

Bi-section algorithm Since the sign of the first derivative changes at a critical point, it is possible to search a critical point if we find two initial points such that the sign of the first derivative is different. The simplest algorithm that uses this property is known as the *bi-section method*, and it can be formulated as this:

Initialization:

1. Choose an acceptable error ϵ
2. Find an initial left point x_L such that $\frac{df(x_L)}{dx} < 0$
3. Find an initial right point x_R such that $\frac{df(x_R)}{dx} > 0$

Iteration:

1. Bisect the range $[x_L, x_R]$, $x' = \frac{x_R + x_L}{2}$,
2. If $\frac{df(x')}{dx} < 0$ then make $x_L = x'$, else if $\frac{df(x')}{dx} > 0$ then make $x_R = x'$,
3. If $x_R - x_L \leq 2\epsilon$ then the solution is $\frac{x_R + x_L}{2}$, else repeat.

That is, we start with two initial values for which the first derivative has different sign. If the function is continuously differentiable, then there must be a value somewhere in between these values where the first derivative is zero. Then, we make a bi-section and continue looking in the part of the range for which there is a change in the sign of the first derivative. The process stops either when we find exactly a critical point or when the distance between both points is less than the acceptable error.

For instance, suppose that your equation $f(x) = 0$ had a solution precisely at $x = \frac{1}{\pi} - 0.3183$. (You don't know that of course; you're trying to find the solution!) You'd first narrow down the interval from $[0, 1]$ to $[0, \frac{1}{2}]$; then to $[\frac{1}{4}, \frac{1}{2}] = [0.25, 0.5]$; then to $[\frac{1}{4}, \frac{3}{8}] = [0.25, 0.375]$; then to $[\frac{5}{16}, \frac{3}{8}] = [0.3125, 0.375]$; and so on. See figure 3 for a visual example.

2.2 Multidimensional problems set-up

For multidimensional problems the logic applied is the same, except that in this case it is necessary to consider partial derivatives in the different decision variables or dimensions of the problem.

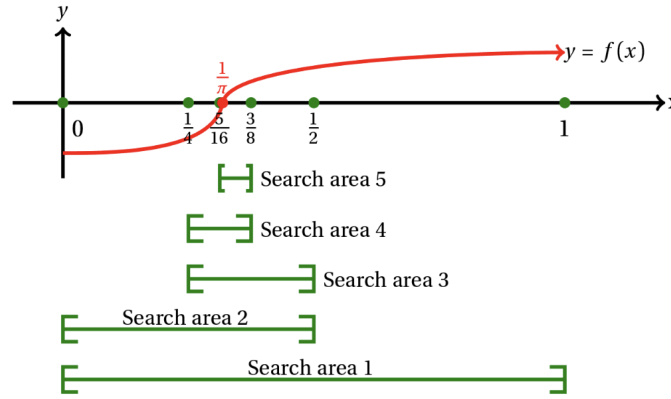


Figure 3: Bi-section example.

Thus, given an n -dimensional function:

$$f(x), x = [x_1, x_2, \dots, x_n],$$

the **gradient vector** is defined as a vector containing the first partial derivatives:

$$\nabla f(x) = \left[\frac{\delta f(x)}{\delta x_1}, \frac{\delta f(x)}{\delta x_2}, \dots, \frac{\delta f(x)}{\delta x_n} \right]^T$$

The gradient represents the slopes of the tangent planes to $f(x)$ in every dimension. For a minimum, the gradient must be zero, as an extension of the first derivative test of one dimensional problems.

1. Gradient test The gradient must be zero at a critical point x^* , $\nabla f(x^*) = [0, \dots, 0]^T$.

To know if the function decreases in every direction around x^* , we need to define the Hessian:

$$Hessf(x) = \begin{bmatrix} \frac{\delta^2 f}{\delta x_1^2} & \frac{\delta^2 f}{\delta x_1 \delta x_2} & \cdots & \frac{\delta^2 f}{\delta x_1 \delta x_n} \\ \frac{\delta^2 f}{\delta x_2 \delta x_1} & \frac{\delta^2 f}{\delta x_2^2} & \cdots & \frac{\delta^2 f}{\delta x_2 \delta x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\delta^2 f}{\delta x_n \delta x_1} & \frac{\delta^2 f}{\delta x_n \delta x_2} & \cdots & \frac{\delta^2 f}{\delta x_n^2} \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1n} \\ h_{21} & h_{22} & \cdots & h_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ h_{n1} & h_{n2} & \cdots & h_{nn} \end{bmatrix}$$

That is, the Hessian is a $n \times n$ matrix that contains the second order derivatives.

To determine how the function increases or decreases in any direction around a given point, we can check *Sylvester's criterion*. This criterion evaluates the *leading principal minors* of the Hessian matrix. A leading principal minor k of a Hessian matrix is the determinant of the square sub-matrix that includes columns and rows from 1 to k , that is, if we note, H_k as the leading minor of the Hessian matrix A , we can compute the leading principal minors as:

$$H_1 = |h_{11}|, H_2 = \begin{vmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{vmatrix}, \dots$$

Note that an n -dimensional function has n Hessian leading minors. Now, the Hessian determinants can help us decide whether a function is a local or global minimum, according to the following definitions:

- *Definitive positive Hessian*: all leading principal minors are greater than zero $H_i > 0, \forall i \in [1, \dots, n]$.
- *Semi-definitive positive Hessian*: all leading principal minors are greater or equal than zero $H_i \geq 0, \forall i \in [1, \dots, n]$.
- *Definitive negative Hessian*: all leading principal minors are lower than zero $H_i < 0, \forall i \in [1, \dots, n]$.
- *Semi-definitive negative Hessian*: all leading principal minors are less or equal than zero $H_i \leq 0, \forall i \in [1, \dots, n]$.

2. Second partial derivative test Now, the second derivative test becomes:

- In a global maximum, the Hessian is definitive negative;
- In a global minimum, the Hessian is definitive positive;
- In a local maximum, the Hessian is semi-definitive negative;
- In a local minimum, the Hessian is semi-definitive positive.

3 Gradient Descent Methods

Gradient Descent Methods constitute a large family of heuristic methods based on the gradient definitions presented in the set-up of unconstrained NLP. Gradient descent methods are useful to find critical points and test whether they are global minimum (for minimisation problems) or global maximum (for maximisation problems).

Basically, they consist of two types functions:

- *search functions*: to find points where the gradient is zero or close to zero, i.e. functions to find critical points.
- *test functions*: to verify whether the critical point is a global or local maximum or minimum.

The different implementations have different performance trade-offs depending on the characteristics of the optimisation problem. Next we will cover two different methods, the simple gradient descent method and the stochastic gradient descent (you probably know these methods from your AI, optimisation courses).

Simple Gradient descent

In the Simple gradient descent method, the algorithm starts at an initial point, calculates the gradient, and if it's different from zero, it looks for the next point to test in the direction opposite to the gradient. Recall that, at any given point x^* the gradient of the objective function $\nabla f(x^*)$ represents a vector tangent to $f(x^*)$. This tangent vector represents the direction of the maximum change of x towards infinity in the n -dimensional space. Therefore, the algorithm continues the search in the opposite direction to this vector, assuming that this will provide the maximum change of the function towards its minimum value. With this, the Simple Gradient descent method can be defined as:

Start:

Choose an acceptable error ϵ

Choose a starting point x'

Iterate:

1. Calculate $\nabla f(x')$,
2. If $|\nabla f(x')| \leq \epsilon$ then stop with x' as the solution, else
3. set $x'' = x' - t \cdot \nabla f(x')$, $t \geq 0$, $t \in \mathbb{R}$ and iterate again.

Note that since t is a positive value and $\nabla f(x')$ is a vector, we are changing the value of x in the direction of the vector. The change is proportional to t , which can be a fixed parameter or a calculated parameter depending on the implementation. One implementation alternative is to use a one-dimensional search (e.g. using a bisection algorithm) to find x^* such that $f(x'')$ is a local minimum. Note that since both x' and $\nabla f(x')$ are vectors, the only unknown is t , and this bisection is a reduction of the multidimensional minimisation problem to a one-dimensional minimisation problem.

Stochastic Gradient Descent (SGD)

The stochastic gradient descent method can be applied in problems where the gradient of the objective function can be estimated from a subset of the parameters needed to compute the closed form of the gradient. The stochastic gradient descent is one of the fundamental optimization methods used in machine learning. Let us consider the following objective function:

$$\min R_n(\theta) = \frac{1}{n} \sum_t^n L_t(\theta).$$

The stochastic gradient descent method is defined as:

Start:

Choose an acceptable error ϵ

Choose a starting point θ , e.g. $\theta = 0$

Choose a learning factor, ζ_k , for each iteration k

Iterate:

1. Select a random value t^*
2. Calculate the gradient of $\nabla L_{t^*}(\theta_k)$
3. Update θ_k in the direction contrary to the gradient at t^* : $\theta_{k+1} = \theta_k - \zeta_k \cdot \nabla L_{t^*}(\theta_k)$
4. Repeat until the difference between two consecutive estimations is sufficiently small, or until the maximum number of iterations is reached.

MME8215: Operations Research

X. Decision Theory and Game Theory (I)

Contents

1 Decision Theory	1
1.1 Payoff Matrix	2
1.2 Dominance	3
1.3 Decision Rules	3
1.3.1 MinMax	4
1.3.2 MaxMax	4
1.3.3 Bayesian	5
2 Decision Trees	5
2.1 Solving a Decision Tree	7
2.2 Imperfect Information	7
2.3 Value of Perfect Information	8

This course provides an introduction to decision theory, where we will focus on solving decision problems. In this case a decision maker must select among various alternatives, the alternatives provide different performance. This course also gives an introduction to game theory, a field dedicated to modelling and solving decision problems involving several rational decision makers. For the game theory part we focus on solving decision problems where the decision maker must select among various alternatives, focusing on competition and collaboration.

1 Decision Theory

Decision theory spans a combination of problem-solving techniques to find the best decision in complex decision problems. We will use the following terminology:

- **Alternatives:** Decision variables which are controllable and depend on the decision maker's decision.
- **Uncertainty and states of nature:** External variables, these variables are uncontrollable and need to be estimated or assumed.
- **Performances:** Profit or Cost (utility) of the result of a decision

The **objective** is to find the alternative with the highest performance for the decision maker, possibly in situations where the performance may depend on external variables which cannot be controlled.

We will be focusing on the following aspects:

- **Impacts over time** All the important consequences of a problem do not occur at the same instant in time.
- **Uncertainty** At the time the decision-maker must select an alternative, the consequences are not known with certainty.
- **Possibility of acquiring information** Often we can acquire additional information to support decision-making at a cost. For instance, to collect seismic information to decide whether to drill for oil. Decision theory provides methods to evaluate if it is worth acquiring additional information or not.
- **Dynamic aspects** The problem might not end immediately after an alternative is chosen but might require further analysis (e.g. further decisions)

1.1 Payoff Matrix

The payoff matrix is a tool to represent the performance of the different alternatives against the possible future outcomes of an uncontrolled event, or states of nature. The **different rows** of the matrix represent the alternatives of the decision maker, and the **different columns** of the matrix, the different possible states of nature. Every cell in the matrix contains the performance of each alternative given the occurrence of each state of nature.

Let us note the decision maker's alternatives as $a_1, a_2, \dots, a_m$, and the different states of nature as $s_1, s_2, \dots, s_n$. Let us note the performance of alternative a_i when s_j has occurred as u_{ij} , then the pay-off matrix is:

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ u_{21} & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ u_{m1} & u_{m2} & \cdots & u_{mn} \end{bmatrix}.$$

Note that the payoff matrix represents all possible outcomes of our decision under uncertainty, given that the states of nature are exhaustive (together they describe all the possible outcomes of the uncontrolled variable) and mutually exclusive (if one occurs, then the rest cannot occur). The payoff matrix can also be represented in tabular form as depicted by table 1.

Table 1: Payoff matrix as tabular.

Decision alternatives	state 1	state2	...	state n
a_1	u_{11}	u_{12}	$\dots$	u_{1n}
a_2	u_{21}	u_{22}	$\dots$	u_{2n}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
a_m	u_{m1}	u_{m2}	$\dots$	u_{mn}

Example 1. Let us, at this point, set one example to be used in the next sections of this course. In the example, the decision maker needs to select a financial product among a set of market alternatives: Gold, bonds, stock options, deposit, or hedge fund. Each product provides different benefits or losses depending on the behaviour of the market. The behaviour of the market is not

controlled by the decision maker, therefore, it is characterised in this example as a set of states of natures:

- *Accumulation Phase*: The market is stable and characterised by a slow raise.
- *Mark-up Phase*: The market has been stable for a while, investors feel secure and the market is characterised by a fast raise.
- *Distribution Phase*: Most investors start selling to collect benefits, the market stabilises and there is little or no change.
- *Mark-down Phase*: Some investors try to hold positions as there is a large fall of the market.

Table 2: Payoff matrix for example 1.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down
Gold	−100	100	200	0
Bonds	250	200	150	−150
Stock options	500	250	100	−600
Deposit	60	60	60	60
Hedge fund	200	150	150	−150

The payoff matrix presented by table 2 shows the expected performance in euros of the investment of each product under each market cycle phase.

1.2 Dominance

Dominance is a property of alternatives that provide a better performance than others for every state of nature. In example 1, it can be seen that the bond provides a higher benefit than the option hedge in any market scenario, therefore the bond option is dominant over the hedge fund option. Dominance allows us to simplify the decision-making process, eliminating the dominated alternatives in favour of the dominant alternatives. Taking into account dominance, the payoff matrix from table 2 can be written as the one in table 3.

Table 3: Payoff matrix for example 1, taking into account dominance.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down
Gold	−100	100	200	0
Bonds	250	200	150	−150
Stock options	500	250	100	−600
Deposit	60	60	60	60

Any rational decision maker would select bonds before hedge fund option under any market phase and therefore, the latter can be ignored.

1.3 Decision Rules

Decision rules are criteria used by a rational decision maker to make systematic decisions, i.e., to select the best alternative. This section describes some of the most important criteria in related literature.

1.3.1 MinMax

The MinMax criteria aims to minimise loss in the worst-case scenario. It is therefore a *conservative*, or pessimistic criteria. Also known as Maximin (minimise the maximum loss), this criteria first finds the minimum value of the performance of each decision variables in every scenario:

$$m_i = \min\{u_{i1}, u_{i2}, \dots, u_{in}\}, \forall i = [1, 2, \dots, m].$$

These values represent the worst case scenario in every decision alternative, then the MinMax criteria selects the maximum of these values:

$$d = \operatorname{argmax}(m_1, m_2, \dots, m_m).$$

The function *argmax* above returns the index of the maximum value rather than the maximum value. Hence, it will return the index of the alternative for which the maximum is value.

For example 1, table 4 with the associated explanations, presents the result of the the MinMax criteria.

Table 4: Payoff matrix for example 1, taking into account dominance and Minmax.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down	m_i
Gold	-100	100	200	0	-100
Bonds	250	200	150	-150	-150
Stock options	500	250	100	-600	-600
Deposit	60	60	60	60	60

$$d = \operatorname{argmax}(-100, -150, -600, 60) = 4, \text{ and } v = \max(-100, -150, -600, 60) = 60.$$

The MinMax criteria models the decisions of a conservative rational decision maker and in this example, a conservative investor will invest in deposits.

1.3.2 MaxMax

The MaxMax criteria on the other hand aims to maximise profit in the best-case scenario. It is therefore an *aggressive*, and optimistic criteria. This criteria first finds the maximum value of the performance of each decision variables in every scenario:

$$o_i = \max\{u_{i1}, u_{i2}, \dots, u_{in}\}, \forall i = [1, 2, \dots, m].$$

These values represent the best case scenario in every decision alternative, then the MaxMax criteria selects the maximum of these values:

$$d = \operatorname{argmax}(o_1, o_2, \dots, o_m).$$

For example 1, table 5 with the associated explanations, presents the result of the the MaxMax criteria.

$$d = \operatorname{argmax}(200, 250, 500, 60) = 3, \text{ and } v = \max(200, 250, 500, 60) = 500.$$

The MaxMax criteria models the decisions of a an *aggressive* and optimistic decision maker and in this example, an optimistic investor will invest in stock options.

Table 5: Payoff matrix for example 1 taking into account dominance and MaxMax.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down	o_i
Gold	-100	100	200	0	200
Bonds	250	200	150	-150	250
Stock options	500	250	100	-600	500
Deposit	60	60	60	60	60

1.3.3 Bayesian

The Bayesian criteria takes into account additional information about the likelihood of each state of nature. It is therefore the criteria applied by an informed decision maker which wants to factor in the decision the likelihood of each alternative. The probabilities of each state or nature as defined as $p(s_j)$. The Expected Monetary Value (EMV) of each alternative is defined as:

$$EMV_i = \sum u_{ij} * p(s_j).$$

The EMV can be interpreted as the weighted average value of the alternative given the probabilities of occurrence of each state of nature.

The Bayesian criteria selects the alternative with the maximum EMV :

$$d = \operatorname{argmax}(EMV_1, EMV_2, \dots, EMV_m).$$

Let us use again example 1 to apply this decision criteria, table 6

Table 6: Payoff matrix for example 1 taking into account dominance and Bayesian.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down	EMV
Probabilities ($p(s_j)$)	0.2	0.3	0.3	0.2	-
Gold	-100	100	200	0	70
Bonds	250	200	150	-150	125
Stock options	500	250	100	-600	85
Deposit	60	60	60	60	60

$$d = \operatorname{argmax}(70, 125, 85, 60) = 2, \text{ and } v = \max(70, 125, 85, 60) = 125.$$

The Bayesian criteria selects the decision alternative with index 2 (Bonds) and its EMV is 125.

2 Decision Trees

Decision Trees are chronological representations of complex decision processes that can be used to find an optimal decision based on a specific criteria, using a divide-and-conquer approach.

Decision trees have three main graphical components:

- **Decision Nodes:** Represent decisions under certainty. Each branch represents a decision alternative and is represented by a utility or cost as a measure of its performance.
- **Random nodes:** Represent possible outcomes under uncertainty. Each branch corresponds to a possible state and is represented by a probability P .
- **Leafs:** Possible outcomes of the decision process. They are represented by the performance, the utility or cost value corresponding to the connected decision nodes and random nodes.

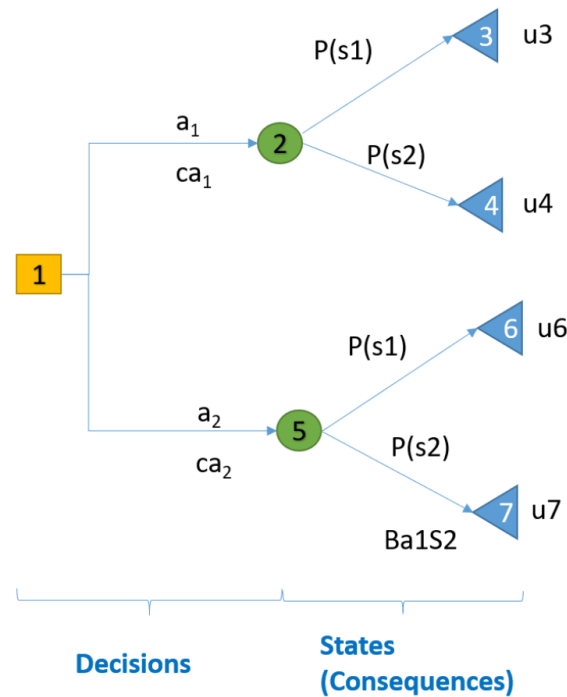


Figure 1: A decision tree.

Figure 1 shows these components and their relationships as a baseline example. The example shows just a decision node, labeled as 1, with two different alternatives, noted as a_1 and a_2 . Each alternative has a different utility value (ca_1 for a_1 and ca_2 for a_2). There is some uncontrolled variable that can impact our decision, and therefore, for each of the output branches of the decision node, there are two random nodes, labelled as 2 and 5, which represent the *uncontrolled* variable. From each node, there are two different branches that represent the two possible states of the decision variable. Each branch is labelled with the corresponding probability of the same state of nature, $P(s_1)$ and $P(s_2)$. Finally, the leaf nodes, are labelled according to the utility values of each decision under each state of nature (u_3, u_4, u_6, u_7). Note that the same information can be represented in a payoff matrix as the one described by table 7.

Table 7: Payoff matrix for decision tree.

Alternatives	state 1	state2
Probabilities	$P(s_1)$	$P(s_2)$
a_1	$u_3 + ca_1$	$u_4 + ca_1$
a_2	$u_6 + ca_2$	$u_7 + ca_2$

The decision tree allows to represent more complex set-ups adding more nodes and uncontrolled variable, even dealing with different combinations of uncontrolled variables and decision nodes at each branch. To define a decision tree, it is necessary to apply the following steps:

1. **Structure the problem:** define decision steps, states, objectives and their performance measures. In general, a decision problem can have multiple objectives. Each objective has a performance measure to represent preferences of the decision maker (e.g. An objective can be to minimize costs and the performance metric can be a monetary unit like euros). If there are multiple objectives, it is necessary to take into account the trade-offs among different

objectives to obtain a single utility function to represent preferences. However, for the rest of the unit, we will deal with single objective decision problems. Utility functions help us quantify the value of each decision. Each alternative can have a positive impact or negative impact on the value.

2. **Describe consequences:** describe the probability of each possible state of nature. In situations of certainty, each decision will have a positive or negative impact in the value of the decision. In situations of uncertainty or risk, each possible state will have a different probability of occurrence. We need an estimation of these probabilities to quantify the value of each decision.
3. **Assign a value to each decision alternative:** Define the utility of each decision. We need to estimate the utility of each possible outcome of the decision problem.
4. **Decide which of the proposed alternatives is preferred:** To decide which of the proposed alternative is preferred, it is necessary to solve the decision tree, as described in the next section.

2.1 Solving a Decision Tree

Decision trees are solved from right to left, that is, from the leaves, back to the initial decision node. At every decision step, it is necessary to quantify the value of each alternative, taking into account the probability of states of nature. Moreover, the values of random nodes and decision nodes are calculated differently.

The value of a random node is calculated using a Bayesian approach, as the weighted expected value across the different states:

$$V_r = \sum_{i=1}^n P(s_i) * V_i.$$

Where $P(s_i)$ is the probability of state i and V_i is the value at branch or leaf i .

The value of a decision node on the other hand is calculated as:

$$V_n = \max\{V_i - ca_k, V_j - ca_p, \dots\}$$

That is, a rational decision would always select the alternative with the highest value across the different alternatives, taking into account their costs.

With this two simple rules, the value of a tree can be calculated from its leaves, no matter how complex the set-up is and regardless of the number of steps.

2.2 Imperfect Information

Sometimes we need to decide whether to incorporate into the decision *imperfect information*. Imperfect information is information about uncontrolled variables that we can incorporate into the decision (possibly at a cost) to improve the performance. For instance, coming back to the previous example, imagine that our decision problem is to decide a financial product. To make a better decision, we may purchase a forecast report of the market trends for the following year. The report forecasts a given market trend, but the state is not known with certainty. It is possible to evaluate the usefulness of incorporating this information into the problem. Applying the *Bayes Theorem*, the probabilities of the different states of random nodes now will be conditioned on the outcome of the imperfect information (in the example, of the forecast report).

In these cases, it is necessary to review the different probabilities. After imperfect information, the probabilities of occurrence of the different states become revised (posterior) probabilities. The revised probabilities are calculated by applying the Bayes Theorem.

Let us assume that there are n different states and that, consequently, the imperfect information may provide additional information about the occurrence of each of the states. The introduction of imperfect information can be modeled first as a decision node that models the decision of whether to incorporate imperfect information or not, and then as a random node in the branch of the imperfect information state i . For instance, let us consider figure 2, a basic example with two alternatives and two possible states.

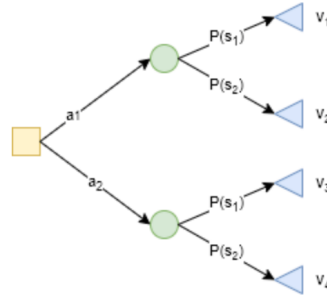


Figure 2: Imperfect information, an example tree.

Let us now introduce the acquisition of imperfect information about the states. First, the decision maker needs to decide whether not to incorporate this information (make the decision without imperfect information) or to incorporate this information (make the decision with additional information, possibly at a cost). The diagram in figure 3 introduces this first decision as an additional decision node. Then, since the result of the forecast is not controlled by the decision maker, its two possible results - i_1 when it forecasts s_1 and i_2 when it forecasts s_2 - are modelled as a random node with two states.

Note that now the probability of state s_j is conditioned to the event i_i in every branch. We can apply the Bayes Theorem to calculate the probability that event s_j occurs given that the imperfect information forecasts i_i :

$$P(s_j|i_i) = \frac{P(s_j) \cdot P(i_i|s_j)}{P(i_i)} = \frac{P(s_j) \cdot P(i_i|s_j)}{\sum_k P(s_k) \cdot P(i_i|s_k)}.$$

In the above example, this yields:

$P(s_1)$ and $P(s_2)$ are the a priori probabilities, or the probabilities without further information. The conditional probabilities: $P(i_1|s_1)$, $P(i_1|s_2)$ refer to the probability that information i_1 occurs under states s_1 and s_2 , respectively. The same applies to $P(i_2|s_1)$ and $P(i_2|s_2)$. This conditional probabilities can be estimated from a historical record of events.

For instance, connecting with our previous example, let us assume that s_1 represents a market raise state and s_2 a market fall state, and a_1 and a_2 two financial products. We may purchase a financial report to guide our investment decision, at a cost. The report has a previous record of predicting 95% of the times a market fall correctly, that is $P(i_1|s_1) = 0.95$ and consequently $P(i_1|s_2) = 0.05$. The record also tells us that the forecast is right 90% of the time when the market falls, therefore $P(s_1|i_2) = 0$ and $P(s_2|i_1) = 0$. With these values, we can calculate the revised probabilities and solve the tree to determine whether to incorporate the report or not.

2.3 Value of Perfect Information

The previous section described how to calculate the value of a decision tree after incorporating additional information, which may come at a cost. An obvious question that arises is, how much should the decision maker be willing to pay for additional information? This quantity is known

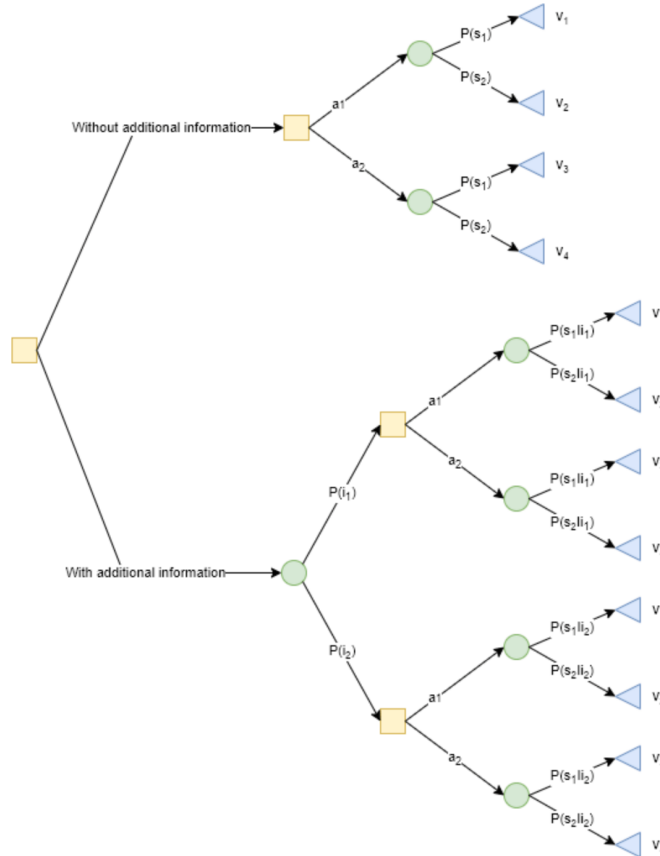


Figure 3: Imperfect information, first decision as an additional decision node.

$$P(i_1) = P(s_1) \cdot P(i_1 | s_1) + P(s_2) \cdot P(i_1 | s_2)$$

$$P(s_1 | i_1) = \frac{P(s_1) \cdot P(i_1 | s_1)}{P(i_1)}$$

$$P(s_2 | i_1) = \frac{P(s_2) \cdot P(i_1 | s_2)}{P(i_1)}$$

$$P(i_2) = P(s_1) \cdot P(i_2 | s_1) + P(s_2) \cdot P(i_2 | s_2)$$

$$P(s_1 | i_2) = \frac{P(s_1) \cdot P(i_2 | s_1)}{P(i_2)}$$

$$P(s_2 | i_2) = \frac{P(s_2) \cdot P(i_2 | s_2)}{P(i_2)}$$

as the Value of Perfect Information. Perfect Information is an abstract concept that represents information that would make the occurrence of a state of nature certain. For instance, a forecast report which can tell us with total certainty which is going to be the trend of the market in the future. Obviously, this type of forecast does not actually exist in reality, but we can model its effect in the decision tree and solve it to calculate its final value. The Expected Value with Perfect Information (EVPI) is the expected final value of the tree given that we had perfect information. It is also an abstract concept to assess the value of the decision-making problem with access to

perfect a priori information.

For instance in the example above, perfect information means that the forecast report is always right, and therefore we can plug in $P(s_1|i_1) = 1$ and $P(s_2|i_2) = 1$ since we know that the information about the states of the report is always right, and since it is always right, then also this means that $P(s_1|i_2) = 0$ and $P(s_2|i_1) = 0$. This simplifies the tree and the value of the node with additional information becomes:

$$EVPI = P(s_1) * \max(v_1, v_3) + P(s_2) * \max(v_2, v_4).$$

Since this is the maximum value of that node provided that we had perfect a priori information, we can compare the *EMV* that we obtain solving the report to the *EVPI* to see how close we are to perfect information. This difference is known as the **Value of Perfect Information** (*PVI*):

$$VPI = EVPI - EMV.$$

MME8215: Operations Research

X. Decision Theory and Game Theory (II)

Contents

1 Game Theory - Introduction	1
1.1 Game examples	3
1.2 Game classification	3
1.3 Describing a game	4
2 Dominant Strategies	5
2.1 How to locate saddle points	7

1 Game Theory - Introduction

The student may have heard that game theory is a mathematical analysis of strategic behavior; and may know that it has found important applications in many fields of study such as warfare, politics, diplomacy, biology, and especially economics.

But what exactly is meant by "analysis of strategic behavior" and what does this have to do with games? The concept of a mathematical game is different from and more general than the usual notion of a parlor game, such as chess, Monopoly, poker, or ticktacktoe. A mathematical game is a special type of optimization problem.

In a game, the outcome depends not only on your decision but also on the decision(s) of other agents over which you have no control. Those other agents could be other people, but they also could be elements of chance or randomness. So you still attempt to make a decision that will lead to the best outcome for you, but you must do so while taking into account what others might do that could affect your outcome. When you make your decisions in a thoughtful way, taking into account the possible actions of others, you are said to be behaving strategically, and those are the situations of mathematical game theory.

Besides parlor games like chess and poker real life examples of games arise naturally. For example,

- in business, if you are deciding between several potential locations for your next franchise store, the eventual success of your new store will at least partly depend on where your competitors open their next stores.
- In warfare, if you are sending supplies and/or reinforcements to your troops located on a remote island, you must choose which supply route to take. Your enemy can potentially thwart your plans – will they attack this route or that? You do not know in advance, but

nevertheless, you must try to anticipate what they might do and what the outcome might be if they do attack. In other words, you must think and act strategically.

- In the next presidential election, if there are three candidates, should you vote for your favorite candidate, who is a long shot, or should you vote for your second-favorite candidate who has a much better chance, just to help make sure the candidate you like the least is not elected? The success of either strategy depends partly on how others vote. Thus, voting can be regarded as a strategic game if there are three or more candidates.

A mathematical game consists of a **set of players** (players can be individuals or any collection of individuals such as teams, corporations, armies, countries, etc.), each with a **collection of actions** open to them, called **strategies**. Once the players select their strategies, the play of the game will result in an **outcome**. We stipulate that each outcome of the game results in a **payoff** for each of the players, which we assume can be measured numerically.

For example, the payoff might be binary in nature – a player is either a winner or a loser, like in ticktacktoe or chess. In this case, we might designate the payoff of 1 to indicate a winner and 0 to indicate a loser. In other games, the payoff might be an integer, such as the number of “points” earned. In some situations the payoff is not easily measured numerically (e.g. what is the payoff for a battle between two armies? number of casualties, difference in casualties?).

Utility theory is that part of game theory that deals with how a player values a particular outcome – the utility of the outcome. It may be quite difficult to assign a specific utility of an outcome. To make matters worse, the same outcome with the same payoff might be valued differently by different people, so the players in a game may not even agree on the utility of an outcome.

To illustrate the difference between payoff and utility, imagine an estate being divided at the reading of a will. One of the heirs might inherit an \$80,000 Rolls Royce, while his brother may inherit \$60,000 cash. Even though the payoff for the car is \$80,000, the person who inherits the car may actually value the \$60,000 cash more – perhaps he doesn’t drive or doesn’t look forward to the bother and expense of maintaining the car. So the utility of the car may actually be less than \$60,000 to the person who inherited it.

We will greatly simplify our treatment of games if we make the following assumption:

Each outcome of a game has a numerical payoff (or value or utility), which is known to all the players before the game starts and upon which all players agree.

We need to make some assumptions:

- all the players have the same information about the game.
- The players will always try to act in a way that will lead to the best possible payoff for themselves.
- Players in a game are skilful, and will act rationally (that is, they will always strive to obtain the best possible payoff for themselves).

We will focus on single-move games (e.g. chess or tick-tack-toe) because they are the easiest to analyse. Thus we make the assumptions:

- *The play of the game consists of each player selecting one of the strategies open to them; then the outcome is reached.*
- *The players will make their selection of strategies simultaneously and independently.*

1.1 Game examples

Example 1. *Two-Finger Morra.* Two players, Ann and Bill, face each other with their right hands hidden behind their backs. On the count of three, they move the hidden hand to the space between them, revealing either one or two fingers extended. If the total number of fingers revealed is odd, Ann pays Bill \$1. If the total number of fingers revealed is even, Bill pays Ann \$1.

Example 2. *Rock Paper Scissors.* Ann and Bill play the well-known game of rock paper scissors, following the usual conventions that rock beats scissors, scissors beat paper, and paper beats rock. They decide to follow the payoff scheme that the loser pays the winner \$1, but there is no exchange of money if the result is a tie.

Example 3. *Saturday Chores at the Anderson Household, (Version I).* Mr. and Mrs. Anderson have three children – Ian, Jon, and Laurie – who will help with certain chores. Mr. Anderson rewards them by paying a total of \$24: \$6 for washing the dishes, \$6 for cleaning the basement, and \$12 for raking the yard. The children can divide up the tasks any way they want, and they can even share tasks (and split the reward two or three ways). However, past experience shows that if the children discuss their preferences ahead of time, they will just get into an argument. To avoid those arguments, Mr. Anderson has the kids write down their job choice or choices secretly on a piece of paper. The only rule is that all the tasks must be done, so if a job is not claimed by one or more of the children, then Mr. Anderson will assign the task as follows: it will be shared equally by all children who do not choose any task, and if all three have chosen something, then it will be shared equally by all three children.

Example 4. *Saturday Chores at the Anderson Household, (Version II).* Several years later, the Anderson children are all older. Ian is off at college, leaving Jon and Laurie to share the chores. However, nobody plays in the basement anymore, so there are only two tasks – raking and dishes. Mr. Anderson has modified the rewards as follows: He'll pay Laurie \$6 if she chooses the dishes, but he wants to encourage Jon to choose that task more often since he tends to avoid it, so if Jon does the dishes, he earns \$8. If they share the job, he pays \$7: \$4 to Jon and \$3 to Laurie. Mr. Anderson does not want to be unfair to Laurie, so he has a similar asymmetry built into the payoffs for raking in Laurie's favor. He'll pay Jon \$12 to do the job himself, and he'll pay Laurie \$15 to do the job herself. If they share the job, he pays \$13: \$7 to Laurie and \$6 to Jon. The rest of the rules are the same: any unclaimed task is assigned to a child who has claimed nothing, or is split between them if both children or neither have claimed a task.

1.2 Game classification

Games are classified into:

- *two-player games* vs. *n-player games* (where n is 3 or more).
- Considering the total payoffs, *zero-sum games* in which the sum of the payoffs for both players is always 0.
- A minor variation of a zero-sum game is a *constant-sum game*, which means that the sum of the payoffs to the players is a constant
- By the game size, i.e. the number of strategies open to each player.

Two person, zero-sum games are the easiest to analyze.

1.3 Describing a game

Let us now narrow our attention to two-person games, and consider two ways to describe and display the important information of the games in a convenient format. The first way is via *tree diagrams* (also known as a *game tree*).

For example, consider the two-finger morra game described in example 1. To represent it select a player (it does not matter which one), and label a node of the tree with his or her name. Draw branches down from this node corresponding to and labelled with the strategies for that player. At the end of each of those branches, put a node labelled with the name of the other player. From each of those nodes, draw further branches down corresponding to and labelled with the second player's strategies. Since the game ends after each of the two players has selected a strategy (that is, a branch of the tree diagram), the ends of these branches are terminal nodes corresponding to the outcomes of the game. We can then label each outcome with its payoff. Figure 1 shows the game representation as a tree.

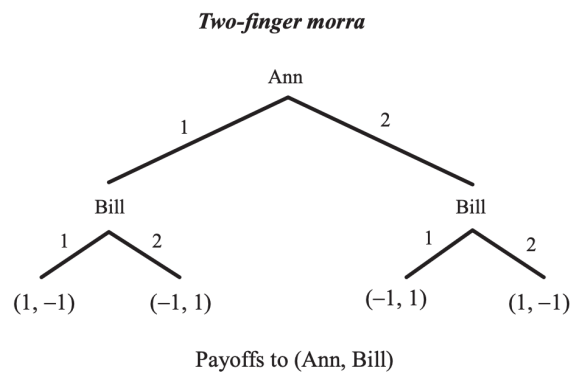


Figure 1: Game tree for two-finger morra.

Since we are assuming that the players make their selections simultaneously it does not matter which player is at the root of the tree. But if we drop the assumption of simultaneous play and look at games where the players move in sequence, then it will be important to draw the game tree with the first player to move at the top, etc.

When a two-person game is depicted by using the game tree, the game is said to be in *extensive form*.

We can make a table of the payoffs so that the rows of the table correspond to Ann's strategies and the columns to Bill's strategies, the following 2×2 matrix, A , called the *payoff matrix* of the game, results.

		Bill	
		1 finger	2 fingers
Ann	1 finger	(1, -1)	(-1, 1)
	2 fingers	(-1, 1)	(1, -1)

Ann, who is called the row player, chooses a row (her strategies), and Bill, the column player, chooses a column (his strategies). The position of the matrix where the row and column intersect is the outcome of the game (more precisely, it is the payoff associated with the outcome).

Since this is a zero-sum game, it suffices to know one player's payoffs (we can easily determine the other, so it is not necessary to use ordered pairs for the payoffs). By convention, we always

display the row-player's payoffs. Thus, for two-person zero-sum (or constant-sum) games, the payoff matrix is an ordinary matrix with numerical entries.

		Bill	
		1 finger	2 fingers
Ann	1 finger	1	-1
	2 fingers	-1	1

In fact, when it is understood which player is the row player and what the strategies are, we omit all the labels on the rows and columns and simply write the payoff matrix as the following:

$$A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

If a game is not constant-sum, the row player's payoffs do not determine the column player's payoffs, so we must retain the ordered pair payoffs as the entries in the matrix. When a **two-person game** is **described** by giving its **payoff matrix**, the game is said to be in **normal form**.

Bellow is the normal form of the game described by example 4. Let Jon be the row player and Laurie the column player.

		Laurie			
		No chore	Dishes	Raking	Both
Jon	No chore	(10, 10)	(12, 6)	(8, 15)	(0, 21)
	Dishes	(8, 15)	(10, 10)	(8, 15)	(4, 18)
	Raking	(12, 6)	(12, 6)	(10, 10)	(6, 13)
	Both	(20, 0)	(16, 3)	(14, 7)	(10, 10)

2 Dominant Strategies

Now, let's see how we can solve a game.

Definition 1 In a two-person game, a strategy S **dominates** a strategy T if each payoff in S is at least as good as the corresponding payoff in T .

In a two-person *constant-sum game*, with payoff matrix A consisting of payoffs to the row player, then Column S dominates Column T if every entry in S is less than or equal to the corresponding entry in T – that is, if $a_{is} \leq a_{it}, \forall i$. On the other hand, row S dominates row T if every entry in S is greater than or equal to the corresponding entry in T – that is, if $a_{sj} \geq a_{tj} \forall j$. We say the **dominance is strict** if at least one of the inequalities is strict.

If you are a player in a two-person game with just two strategies, S and T , and S dominates T , you should clearly play S . If there are more than two strategies and S dominates T , it is not necessarily the case you should play S – you would need to know how S compares to these other strategies besides T . However, one thing is clear: you should not play T . You would always be at least as well off, if not better, by playing S . Therefore, **in any two-person game, rationality dictates that you should never play a dominated strategy**.

Definition 2 In any game (two-person, n -person, constant-sum or variable-sum), an out-come of the game is said to be a **Nash equilibrium** if it has the property that no player stands to gain by a unilateral change in strategy.

Considering the payoff matrix for the game illustrated by example 4, we observe that the largest entries for the first player are all in the bottom row, and the largest entries for the second player are all in the rightmost column. Thus, the strategy of volunteering for both chores dominates all three of the other strategies. So in this game, both players have a dominant strategy. Also, the outcome where these dominant strategies intersect is a Nash equilibrium, (10, 10).

Does this constitute a solution to this game? After all, a Nash equilibrium that results from playing two dominant strategies seems especially stable. For a zero-sum game, yes. As this example is a variable sum game, it can have some subtle dynamics. Variable-sum games are games of partial conflict and that it may be possible by partial cooperation to find outcomes in which both players improve their payoffs (the win-win situations that do not happen with constant-sum games).

In this case, there is no such improvement that is possible (under the assumptions of simultaneous, rational play with no communication between the players and in which the players truly want to maximize the money they make), because by inspecting the payoff matrix, we find no payoffs for both players larger than or equal to 10 simultaneously, except the other payoffs of (10, 10) that all lie on the main diagonal. Thus, each player can see that no further improvements over the (10, 10) payoff are possible for both players at once; any improvement for one player results in the loss for the other player (similar to what happens in a constant-sum game).

Therefore, in this game, it may seem to be safe to say that we have an acceptable solution to the game (although we will not formally discuss variable-sum games until a later chapter). Rational play dictates that both players should volunteer for both jobs, resulting in a payoff of \$10 for each of the children.

Actually, it may be possible to find other outcomes where the payoff to both players is more than \$10 if we use *mixed strategies*,

Definition 3 *The Solution of a two-person game consists of the optimal strategy for each player, as well as the value of the game.*

Definition 4 *Let A be the payoff matrix of a constant-sum game. A saddle point of A is an entry of A that is simultaneously the smallest entry in its row and the largest entry in its column.*

For example, consider the payoff matrix A below where a_{11} is a saddle point. The entry 2 there is the largest entry in its column and is (tied for) the smallest in its row.

$$A = \begin{bmatrix} 2 & 2 \\ 1 & 3 \end{bmatrix}$$

Definition 5 *In the payoff matrix A for a constant-sum game, an entry corresponds to a Nash equilibrium if and only if it is a saddle point.*

Any constant-sum game (in particular, any zero-sum game) in which the payoff matrix has a saddle point is said to be strictly determined.

In a strictly determined game, both players should choose the strategy that leads to the saddle point outcome. Thus, the solution to a strictly determined game is easy to find – choose a strategy containing the saddle point, and the value of the game (for the row player) is the value of the saddle point.

In a strictly determined (constant-sum) game, we can regard the value v of the game as a sort of guaranteed minimum payoff for each player. If the players play optimally, the result will be the saddle point outcome and each player will obtain the value guaranteed there (v for the row player and $c - v$ for the column player, if the payoffs sum to c .) However, if either player deviates from his or her optimal strategy, it may be possible for the other player to have an even better payoff.

2.1 How to locate saddle points

Let us describe an alternate method of locating saddle points. Since a saddle point is the smallest entry in its row, the first step is to record the minimum of each row. Also, since a saddle point is the highest entry in its column, the next step is to record the maximum in each column. Then find the maximum of the row minima (which is the best of the worst-case scenarios for the row player) and the minimum of the column maxima (the best of the worst-case scenarios for the column player. If these values are the same, then the outcome(s) where this occurs will be a saddle point (or saddle points). Suppose the payoff matrix is the following:

$$A = \begin{bmatrix} 1 & -2 & 0 & -1 \\ 3 & 4 & 3 & 6 \\ 5 & 1 & -1 & -4 \end{bmatrix}.$$

We record the row minima and column maxima:

$$A = \begin{array}{ccccc} & & & & \text{Row minima} \\ & \begin{bmatrix} 1 & -2 & 0 & -1 \\ 3 & 4 & 3 & 6 \\ 5 & 1 & -1 & -4 \end{bmatrix} & & & \begin{array}{l} -2 \\ 3 \\ -4 \end{array} \\ \text{Column maxima} & \begin{array}{l} 5 \\ 4 \\ 3 \\ 6 \end{array} & & & \end{array}$$

The largest row minima is 3 (second row), and the smallest column maxima is also 3 (third column), which indicates that the entry $a_{23} = 3$ is a saddle point, which is easily verified.

Consider the following 4×6 payoff matrix, which we may as well assume represents the payoffs in a zero-sum game:

$$A = \begin{bmatrix} 1 & \underline{-2} & 0 & -1 \\ \underline{3} & \underline{\underline{4}} & \underline{\underline{3}} & \underline{\underline{6}} \\ \underline{5} & 1 & -1 & \underline{-4} \end{bmatrix}$$

All four of the entries that are boxed are saddle points. This illustrates that a strictly determined game can have more than one saddle point (i.e., Nash equilibrium). The solution to this game is that the row player should choose either the second or the fourth row, the column player should choose either the second or fifth column, and regardless of these choices, the outcome is at one of the four saddle points. The value v of the game is the common value 1 of all the saddle points.

Theorem 1 *Any two Nash equilibria in a two-person constant-sum game are equivalent and interchangeable.*

MME8215: Operations Research

XI. Decision Theory and Game Theory (III)

Contents

1 Mixed-Strategy Constant-Sum Games	1
1.1 Mixed Strategies, Probabilities, and Expected Payoffs	2
2 Solving Mixed-Strategy games: the Minimax Theorem in the 2x2 case	4
2.1 Derived game	4

1 Mixed-Strategy Constant-Sum Games

Let us solve another strictly determined game that is infinite, the ideas behind it will be useful for solving 2×2 games that are not strictly determined.

Example 1. Antares vs. Bellatrix Antares Industries and Bellatrix, Inc., are the only two companies that sell a certain computer chip, the Pentangle 5.1. There is an estimated market for 400,000 of these chips in the year 2025. Each company must set its prices. Production costs dictate that the price must be at least 80\$ per chip, while industry regulations dictate that the price should be no more than 120\$ per chip. Let x be the price set by Antares and y the price set by Bellatrix. Based on reliable market studies of which both companies are aware, it has been determined that the percentage of the total market which goes to Antares depends on x and y and is reasonably approximated by the function given by

$$E(x, y) = 330 - 3x - 2.7y + 0.003xy.$$

What are the prices x and y that each company should choose, if their goal is to obtain the largest-possible market share?

This can be regarded as a two-person constant-sum game. The strategies for each company are the prices they set (x and y). The values of these variables must be in the real interval $[80, 120]$, so each player has an infinite number of strategies. Once x and y are chosen, then Antares receives a payoff of $E(x, y)$ while Bellatrix receives a payoff of $100 - E(x, y)$.

If the game were finite, we would construct the payoff matrix and search for a saddle point in the matrix. Since we have infinitely many choices for x and y , it does not make sense to construct a matrix, but it should be clear that the function $E(x, y)$ serves the exact same purpose as the payoff matrix - it gives, for each choice (x, y) , the payoff $E(x, y)$ to the row player Antares.

The graph of $E(x,y)$ is a surface, we are concerned with the part of the surface which is defined for $x, y \in [80, 120]$. The Nash equilibrium here is the same as a saddle point for a payoff matrix: a choice of (x, y) for which $E(x, y)$ is simultaneously a minimum in its “row” (being in a row means holding y fixed) and a maximum in its “column” (which means holding x fixed) will represent an outcome of the game where neither player stands to gain by a unilateral change in strategy. For this surface the saddle point is $(x, y) = (90, 100)$ and $E(90, 100) = 60$ for which $80 \leq x, y \leq 120$.

We can also find the revenue for each company, assuming a total market of 400000 chips, the revenue for Antares is $400000(\frac{E}{100})x$ and for Bellatrix is $400000(\frac{100-E}{100})y$.

1.1 Mixed Strategies, Probabilities, and Expected Payoffs

In a game without a saddle point knowing ahead of time what your opponent will do would affect your strategic decision. For example, in *rock paper scissors*, if you knew before the play that your opponent will choose rock, it is clear that you would then choose paper in order to win, etc. In a game like this, secrecy is important. To be successful in rock paper scissors (and other games without saddle points), one way to keep your opponent guessing about your intentions would be to play different strategies in different plays of the game. Sometimes play rock, other times paper, and other times scissors. This is what is meant by a **mixed strategy**, as opposed to a pure strategy, which means you would always play that strategy no matter how often the game is played.

The first task we face is to devise some notation for these mixed strategies. We do so for the row player by writing a row vector $p = (p_i)$, whose i th component p_i is the relative frequency with which the row player plays the i th row. Similarly, for the column player we write a column vector $q = (q_j)$, whose j th component q_j is the relative frequency with which the column player plays the j th column in repeated play of the game. For example, in rock paper scissors (where the strategies are listed in that order), we might have

$$p = [0.3, 0.2, 0.5], \quad q = \begin{bmatrix} 0.4 \\ 0.4 \\ 0.2 \end{bmatrix},$$

which can be interpreted as the mixed strategies where the row player plays rock 30% of the time, paper 20% of the time, and scissors 50% of the time, and the column player would play rock and paper 40% of the time each and scissors the remaining 20% of the time.

The entries in a mixed strategy are nonnegative fractions whose sum is 1, since the percentages must all sum to 100%. Also note that a pure strategy can also be written in this form – for example, the unit vector $p = (1, 0, 0)$ is the pure strategy of “always play rock.” We view the entries of the mixed strategies as *probabilities*.

Example 2.

Consider the following payoff matrix for a zero-sum game. It does not matter what the actual game is; if you like, you can think of it as a version of two-finger morra, where we have modified the payoffs:

$$A = \begin{array}{cc} & \begin{matrix} S & T \end{matrix} \\ \begin{matrix} S \\ T \end{matrix} & \begin{bmatrix} 2 & -2 \\ -4 & 7 \end{bmatrix} \end{array}.$$

It is easy to check that this game is not strictly determined – there is no saddle point. Under repeated play, using mixed strategies, we’d like to compute the expected payoffs for each player.

Suppose that the row player decides to use the mixed strategy $p = [0.2, 0.8]$, and the column player decides to use the mixed strategy $q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$. If we think of each play of the game as a probability experiment, we can compute the expected payoff for the row player as follows. First, observe that there are four possible outcomes – SS , ST , TS , and TT – depending on what strategies each player chooses in any given play. We already know that the row player chooses row S with probability 0.2 (based on the mixed strategy p), and the column player chooses column S with probability 0.6 (based on the mixed strategy q). So what is the probability of the outcome SS ? In order for SS to occur, both players must choose S : In other words, the outcome SS is the intersection of the two events “row chooses S ” and “column chooses S .” Can we compute the probability of this intersection?

If we have two independent events E and F , the probability that they both happen is:

$$P(E \cap F) = P(E) \cdot P(F).$$

The players choose their strategies simultaneously and independently. Therefore, we are justified in multiplying the individual probabilities to obtain the probability of the intersection: $P(SS) = (0.2) \cdot (0.6) = 0.12$. We can compute the probability of each of the four outcomes in the same manner. Also, the values v_i that go with the outcomes are precisely the entries of the payoff matrix. We are now prepared to compute the expected payoff to the row player. We construct a table in Table 1.

Table 1: Expected payoffs in a mixed-strategy game.

Outcome	p_i	v_i	$p_i v_i$
SS	$(0.2)(0.6) = 0.12$	2	$(0.2)(0.6)(2) = 0.24$
TS	$(0.8)(0.6) = 0.48$	−4	$(0.8)(0.6)(−4) = −1.92$
ST	$(0.2)(0.4) = 0.08$	−2	$(0.2)(0.4)(−2) = −0.16$
TT	$(0.8)(0.4) = 0.32$	7	$(0.8)(0.4)(7) = 2.24$

The expected payoff to the row player is

$$E = 0.24 + (−1.92) + (−0.16) + 2.24 = 0.4.$$

On average, the row player will win 0.4 per game, if the players stick to these strategies.

This payoff E depends on p and q , so we denote it $E(p, q)$. We see that the calculation for E is exactly the same as the matrix product:

$$E = E(p, q) = pAq = [0.2, 0.8] \begin{bmatrix} 2 & -2 \\ -4 & 7 \end{bmatrix} \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix} = 0.4.$$

The itemized calculation (this time, of expected payoff in a game) is made simple and automatic via matrix multiplication. If one repeats this exercise for any payoff matrix, regardless of size, we find, in general, that the same result holds.

Proposition 1 *Let A be the payoff matrix of any two-person, constant-sum game. Suppose the row player decides on the (pure or mixed) strategy p and the column player decides on the (pure or mixed) strategy q . Then, in repeated play, the expected payoff to the row player is the matrix product:*

$$E(p, q) = pAq.$$

2 Solving Mixed-Strategy games: the Minimax Theorem in the 2x2 case

Given a constant-sum game that is not strictly determined, is there an optimal way to play it, and if so, how do we find the optimal strategy?

We consider two cases – when we know what strategy our opponent is using and when we don't.

2.1 Derived game

We consider only 2x2 games in the case when we have no idea of what strategy our opponent might use. Suppose a zero-sum game has the payoff matrix:

$$A = \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}.$$

This matrix has no saddle point. The row player, instead of simply considering the two rows as her strategies, thinks of a strategy as an ordered pair of fractions like $p = (0.2, 0.8)$, where we interpret this to mean she plays the first row with probability 0.2 and the second row with probability 0.8 on any one play of the game, and similarly for the column player. We call these ordered pairs mixed strategies. Furthermore, we have seen that once the row player has settled on a mixed strategy to use – say, $p = (0.2, 0.8)$ – and once the column player has also selected his strategy – say, $q = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$ – then we can find the expected payoff to the row player as:

$$E(p, q) = pAq = [0.2 \ 0.8] \begin{bmatrix} 4 & -1 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = -0.6.$$

We have observed that different choices of p and q lead to different expected payoffs to the row player. We are now interested in the question of how each player should choose his or her mixed strategies, p and q , which would lead to the best-possible expected payoff for himself or herself.

Definition 1 *Let A be the payoff matrix for a two-person, zero-sum game. We consider a new game, called the derived game of A , to be the game where the (pure) strategies are precisely the mixed strategies p and q in the original game, and where the payoff $E(p, q)$ to the row player for a particular outcome of the derived game is given by the formula:*

$$E(p, q) = pAq.$$

Remark 1 *The definition of the derived game applies to matrices of any size. Moreover, although we have only stated the definition for zero-sum games, the concept extends to constant-sum or even variable-sum games without change.*

Remark 2 *Since the expected payoff to the column player is just the negative of $E(p, q)$, it follows that the derived game is zero-sum if A is. Similarly, if A represents a constant-sum game, then the derived game is also constant-sum (with the same constant), and finally, if A represents a variable-sum game, the derived game will also turn out to be variable-sum.*

Remark 3 *Since there are an infinite number of mixed strategies (that is, infinitely many different possible choices for p and also for q), the derived game is infinite, even though A is finite.*

Since the derived game is infinite, we cannot write down a payoff matrix. However, sometimes an infinite game can be solved. Let's look more carefully at a mixed strategy $p = (p_1, p_2)$. Since the p_i values are probabilities whose sum is 1, we can rewrite such a mixed strategy in a slightly simplified form, taking $p_1 = x$ and $p_2 = 1 - x$ as $p = (x, 1 - x)$, where $0 \leq x \leq 1$. Similarly, the mixed strategy $q = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$ can be rewritten as $q = \begin{bmatrix} y \\ 1 - y \end{bmatrix}$, where $0 \leq y \leq 1$. So the expected payoff $E(p, q)$, in the case of a 2x2 matrix A , really only depends on the two parameters x and y . Let us abuse the notation a bit and call it $E(x, y)$ in this case. And

$$\begin{aligned} E(x, y) &= E(p, q) = pAq \\ &= \begin{bmatrix} x & 1-x \end{bmatrix} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} y \\ 1-y \end{bmatrix} \\ &= 10xy - 4y - 3x + 1. \end{aligned}$$

The function $E(x, y)$ is what replaces the payoff matrix, since the players choose x and y so as to make $E(x, y)$ as beneficial to themselves as they can. This function is a saddle surface, so the derived game is strictly determined.

If we write $E(x, y)$ in standard form:

$$\begin{aligned} E(x, y) &= 10xy - 4y - 3x + 1 \\ &= 10(xy - 0.3x - 0.4y) + 1 \\ &= 10(x - 0.4)(y - 0.3) - 0.2. \end{aligned}$$

We see that the saddle point is located at $(x, y) = (0.4, 0.3)$, and E has the value -0.2 there.

$x = 0.4$ means $p = [0.4 \ 0.6]$ and $y = 0.3$ means $q = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$.

Since this represents a saddle point, it is a **Nash equilibrium** in the derived game. Thus, as long as both players play these strategies, the payoff to the row player is $v = -0.2$. As with any Nash equilibrium, if a player unilaterally deviates from the optimal strategy, it is possible for the payoff for the other player to improve (and, therefore, the first player's payoff would decrease since it is a constant-sum game).

Since the expected payoff $E(x, y)$ gives the long-term average payoff in the original, finite game, played with mixed strategies instead of pure strategies, these statements also hold for the original game. The choice of mixed strategies $\hat{p} = [0.4 \ 0.6]$ and $\hat{q} = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$ will result in an average payoff of $v = -0.2$ per game to the row player in the long run. If either player deviates from this unilaterally, he or she cannot improve his or her payoff, so we think of $v = -0.2$ as the value of the game, and this solution represents a Nash equilibrium in the original game under repeated play. ($\hat{p}$ indicates an optimal strategy)

Theorem 1 *Let A be the payoff matrix for any 2×2 constant-sum game. If the game is strictly determined, then we already know the solution corresponds to a saddle point. If the game is not strictly determined, then the derived game of A is strictly determined. In particular, in either case, there are optimal strategies $\hat{p}$ and $\hat{q}$ and a real number v (called the value of the game) such that:*

1. $E(\hat{p}, q) \geq v$ for any q ,
2. $E(p, \hat{q}) \leq v$ for any p ,
3. $(\hat{p}, \hat{q}) = v$.

Remark 4 *It can be shown that if A is strictly determined, then $E(x, y)$ is still a saddle surface. However, the saddle point of this surface will be outside the domain $0 \leq x \leq 1$ and $0 \leq y \leq 1$, so the global saddle point of $E(x, y)$ does not correspond to a mixed strategy since x and y are not probabilities outside this domain. Nevertheless, one of the corners of the patch of $E(x, y)$ when $0 \leq x, y \leq 1$ still has the property that (1) – (3) hold. Since the corners correspond to $x = 0$ or 1 and $y = 0$ or 1 , and these correspond to pure strategies in A , we recover the saddle point solution to A in this case. Thus, the statement that the derived game of A is strictly determined always holds, whether or not A is strictly determined. We explore some aspects of this in the exercises.*